UNIVERSITY OF MARYLAND

Master's in Telecommunication Engineering

Department of Electrical & Computer Engineering

ENTS 749E: Network Automation

DESIGN AND IMPLEMENTATION OF MPLS USING ANSIBLE

Submitted by:

Sangeetha Kolla (UID: 121334299)

Sri Chandraja Reddy Allala (UID: 121294654)

Course Instructor:

Dr. Zoltan Safar

TABLE OF CONTENTS

1. INTRODUCTION

IPv4 has long served as the foundation of modern networking, allowing data to be divided into packets and delivered across interconnected systems. However, as data travels through each router, traditional IPv4 forwarding relies on hop-by-hop routing decisions based on the destination IP address. This process involves de-encapsulation at Layer 3 (Network Layer), where each packet is inspected and then forwarded, a method that becomes inefficient as networks scale.

At a deeper level, the process moves from Layer 7 (Application) down to Layer 2 (Data Link), eventually becoming a frame ready for transmission over the medium. As traffic volumes grow and diverse services demand predictable performance, traditional IP routing struggles with latency, scalability, and convergence issues.

To overcome these limitations, Multiprotocol Label Switching (MPLS) was introduced. MPLS inserts a small shim header between Layer 2 and Layer 3, allowing packets to be forwarded based on labels rather than IP lookups, effectively turning routing into fast label switching. This not

only improves performance and efficiency but also enables advanced capabilities like traffic engineering, fast reroute, and scalable Layer 3 VPNs.

For Internet Service Providers (ISPs), MPLS is particularly beneficial. It allows them to deliver isolated services for multiple customers over a shared core without exposing routing details. By moving complexity to the edges (PE routers) and simplifying the core (P routers), ISPs can scale massively, manage overlapping IP spaces, and support high availability and fast convergence — all of which are critical for carrier-grade networks.

This project focuses on setting up and automating the remote configuration and validation of a basic MPLS network using Ansible. The main objective of this project is to set up the necessary protocols and connectivity between network nodes, allowing seamless communication across an MPLS-enabled infrastructure. By automating these tasks the project simplifies management and enhances consistency in network operations.

Setting up an MPLS network involves a lot of repetitive and detailed configuration, which can easily lead to mistakes when done manually. The idea behind this project is to simplify this process using Ansible. Automating this setup can save time, reduce errors, and make it easier to manage and test the network, especially when working with more complex topologies.

2. OBJECTIVE

The main aim of this project is to automate network configuration using Ansible, eliminating the need for manual CLI setup. Automation not only saves time but also reduces the chances of human error, ensuring consistency across network devices.

To achieve this, several key objectives were undertaken. First, a network was designed and IP addresses were assigned to every router and interface, laying the groundwork for connectivity. Next, OSPF (Open Shortest Path First) was configured as the interior gateway protocol to enable dynamic routing and seamless communication between routers.

In addition to routing, stateful firewall rules were implemented to allow essential traffic and protocols through each router's trust zone, enhancing security. MPLS (Multiprotocol Label Switching) was enabled, and LDP (Label Distribution Protocol) was configured on the appropriate interfaces to support efficient data forwarding.

Finally, the complete functionality of the setup was verified to ensure it met all performance and

connectivity requirements. These tasks collectively form the foundation of the automated MPLS network setup, which is explored in detail in the following sections.

3. NETWORK DESIGN AND IMPLEMENTATION

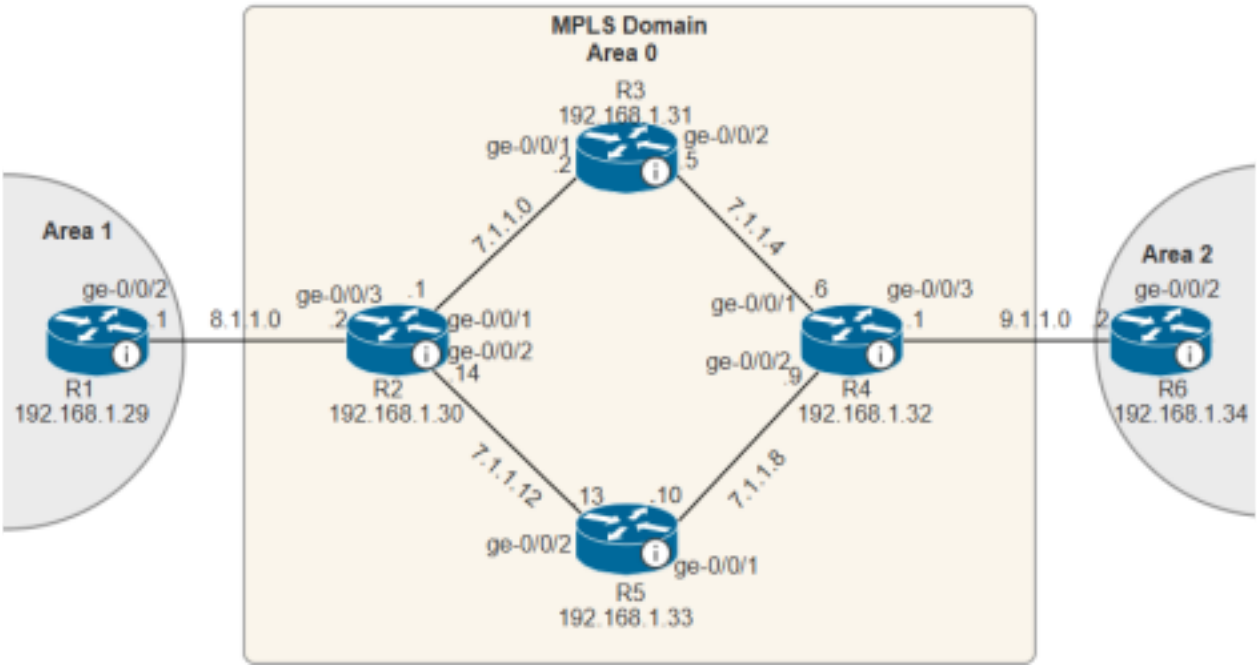


Diagram-1: Network Topology

The above diagram shows the network topology we used for this project using the juniper routers. In this setup scalable, efficient, and secure communication is supported between two ip networks .i.e. Area 1 and Area 2 across a central MPLS backbone.

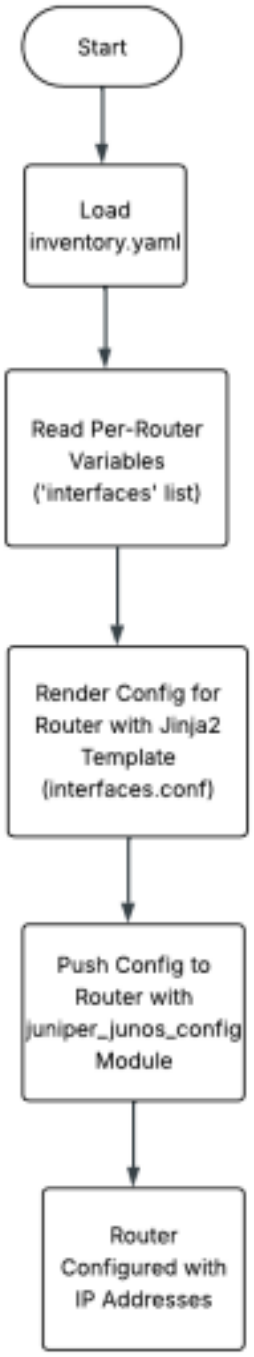
The above network consists of six routers . The router R1 in Area 1 is assigned the IP address 192.168.1.29 and the router R6 in Area 2 assigned the IP address 192.168.1.34, the core MPLS domain in the Area 0 consists of four routers R2, R3, R4 and R5 with Ip addresses 192.168.1.30, 192.168.1.31, 192.168.1.32, 192.168.1.33. Each link between the routers is also assigned a unique /30 subnet. The following routing table shows the interface IP address assignment in detail.

Table-1: Router Details

Router	Interface	Interface IP
R1	ge-0/0/1	8.1.1.1
R2	ge-0/0/1	8.1.1.2
	ge-0/0/2	7.1.1.14
	ge-0/0/3	7.1.1.1
R3	ge-0/0/1	7.1.1.2
	ge-0/0/2	7.1.1.5
R4	ge-0/0/1	7.1.1.6
	ge-0/0/2	7.1.1.9
	ge-0/0/3	9.1.1.1

R5	ge-0/0/1	7.1.1.10
	ge-0/0/2	7.1.1.13
R6	ge-0/0/1	9.1.1.2

3.1 Interface assignment using Ansible



Flow chart-1: Flow chart showing Router Configuration 6

Now as the first step of the automation process, the IP addresses are assigned to the interfaces and loopbacks of all routers using Ansible. This is done using a Jinja2 template file called the *interfaces.conf*, which defines the interface structure for each router. The interface configuration blocks are generated based on the variables provided for each router. The inventory file named *inventory.yaml* in this project defines all the routers involved, along with their IP addresses and authentication credentials in a structured list named interfaces. Ansible uses this list to render device specific configurations that are then pushed to the routers using the juniper_junos_config module over NETCONF. As a result, each router receives the correct IP settings for all its physical and loopback interfaces, forming the foundation for OSPF routing and MPLS label distribution.

4. CONFIGURING OSPF AND SECURITY ZONES

Flow

Chart-2: Configuration of OSPF and security zone setup

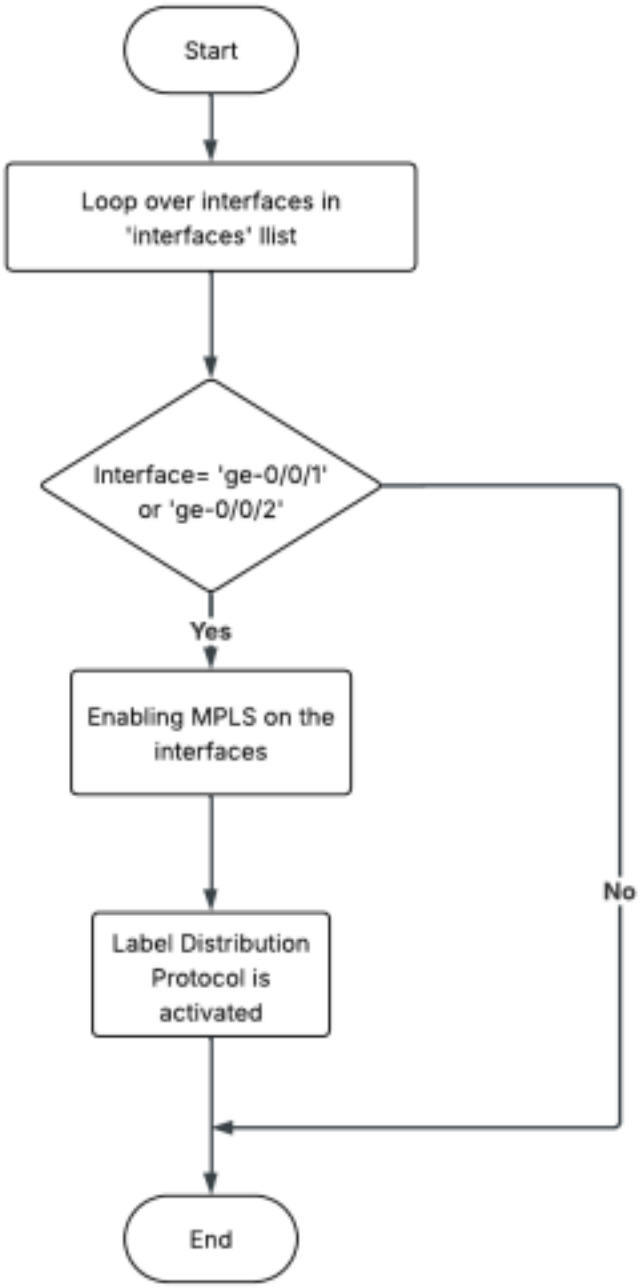
In this project , OSPF (Open Shortest Path First) is used as the interior Gateway Protocol (IGP)

to exchange IP routing information between routers in the network. It plays an important role in preparing the network for MPLS.

We automated OSPF configuration using a jinja2 based Ansible template *ospf_template.conf* applied to each router. This template loops through all router interfaces defined in *inventory.yaml,* assigns each to its correct OSPF area, and configures the routing behavior accordingly. The loopback interfaces are marked as passive to advertise their IPs without forming neighbor relationships, a required setup for LDP, which relies on loopback reachability to establishLabel switched Paths.

To ensure that these control-plane protocols like OSPF and LDP could function properly, the configuration also added all interfaces to a juniper security zone. Inbound traffic was explicitly permitted globally and per interface allowing critical services such as OSPF, LDP, and SSH to pass through the router's stateful firewall. Together these configurations ensured both routing stability and secure protocol communication, forming the backbone of the automated MPLS deployment.

5. MPLS AND LDP CONFIGURATION SETUP



Flow chart-3: MPLS and LDP configuration

As part of the automation process, the next step is to enable MPLS and configure LDP on the

core-facing interfaces of each router using Ansible. This is done through a jinja2 template file named *mpls_template.conf* , which dynamically generates the required configuration based on each router's interface list. MPLS is activated on interfaces such as ge-0/0/1 and ge-0/0/2 by applying the family mpls configuration, allowing them to carry labeled traffic.

Simultaneously, LDP is enabled on these interfaces, as well as on the loopback interface, to facilitate the dynamic exchange of labels between routers. The final configuration is pushed to the devices using the juniper_junos_config module over NETCONF. This step establishes the MPLS Control plane and sets up label-switched paths across the network, enabling efficient and scalable packet forwarding.

6. ANSIBLE PLAYBOOK LOGIC AND IMPLEMENTATION Flow

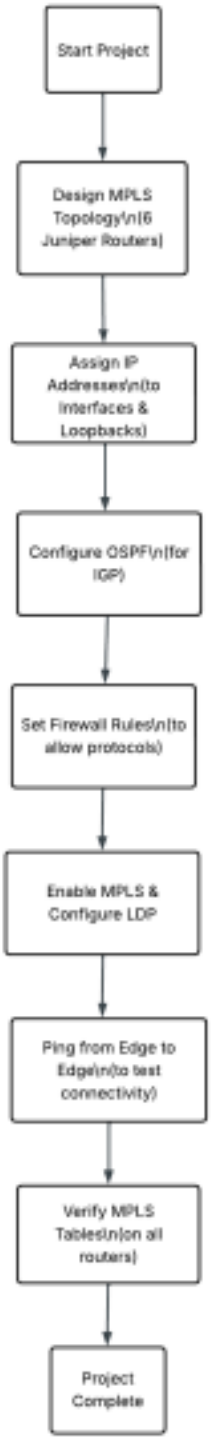


Chart 4: Flowchart illustrating the complete workflow of the project.

To coordinate the whole configuration process, a centralized Ansible playbook named *project01.yaml* was created. This playbook serves as the central coordinator layer that applies all

Jinja2 based configuration templates to the routers. It targets all the devices listed under the routers group in the inventory file and executes tasks over a *network_cli* connection, ensuring compatibility with juniper devices.

The playbook includes structured tasks to configure MPLS on selected interfaces, set up OSPF dynamically by checking each interface's assignment, and enable LDP across both physical and loopback interfaces.

Each of the above tasks uses conditional Jinja2 logic to ensure configurations are applied only to relevant interfaces. This method avoids hardcoding and enhances reusability scalability. By sequentially processing the configuration roles, the playbook ensures that all routers are consistently and correctly configured, laying the groundwork for a fully operational MPLS network with minimal manual intervention.

7. VERIFICATION AND TESTING OF THE MPLS NETWORK

Alongside the automation and configuration of the MPLS network, the verification was also done to ensure the network was operating correctly and the automation is done correctly. This confirmed the success of the project by checking the integrity of routing, MPLS functionality, and end to end connectivity. The following were done for the validation:

Routing table inspection: Each router's routing table was extracted and saved and reviewed to ensure that all loopback and interface networks were correctly learned via OSPF, with appropriate next hop metrics.

MPLS Route Verification: The *mpls-routes.txt* files for core routers were examined to confirm that MPLS forwarding entries were present. This validated that labeled packets would be correctly switched through the MPLS domain.

LDP neighbour checks: Output files like *ldp-neighbours.txt* for each router showed successful label distribution protocol sessions between routers. This ensures that each router has established the expected LDP neighbor relationships.

Inet.3 Table Checks: Verification files such as inet3-routes.txt were used to validate loopback resolution over MPLS, a critical requirement for establishing LSPs. The presence of remote loopback entries confirmed that the control plane was functioning as intended.

Ping Tests and Connectivity: End-to-end connectivity from R1 to R6 was confirmed through successful ICMP ping tests. This helped to verify that the packets were correctly forwarded across the MPLS backbone from Area 1 to Area 2.

Ansible Playbook Validation: The central playbook *project01.yaml* was verified through Ansible logs and device states to ensure all configuration tasks were applied as expected. This ensured that automation was reliable and accurate.

This comprehensive verification process validated both the control and data planes of the MPLS network and confirmed that the automated deployment was successful and production-ready.

8. FUTURE SCOPE

While the current implementation focuses on automating MPLS network configuration using OSPF and LDP, there is considerable scope to extend this project to emulate more complex and realistic service provider environments.

A key area for enhancement is the integration of BGP (Border Gateway Protocol), especially External BGP (eBGP), to enable connectivity between multiple autonomous systems. This would allow for the simulation of inter-domain routing, supporting communication across different service provider networks or between customer and provider infrastructures.

Expanding the OSPF topology to include multiple areas would further enrich the network design, promoting scalability and efficient route management. The deployment of Area Border Routers (ABRs) and a defined backbone area (Area 0) would introduce hierarchical routing, mirroring the architectures used in larger enterprise and ISP networks.

Another significant advancement involves configuring Provider Edge (PE) routers to support Layer 3 VPNs, showcasing MPLS's full potential in delivering secure, scalable customer segmentation. Additionally, incorporating Route Reflectors and Route Targets in BGP would enhance control over route distribution and simplify large-scale VPN management.

Together, these future enhancements would transform the project into a comprehensive service provider-grade network model, offering a deeper understanding of advanced MPLS and routing technologies.

9. CONCLUSION

This project successfully demonstrated the automated configuration and testing of an MPLS network using Ansible. By leveraging Jinja2 templates and structured playbooks, we configured interface IP assignments, OSPF for dynamic routing, and MPLS with LDP across core routers. All configurations were deployed remotely using the juniper_junos_config module over NETCONF. Verification steps, including routing table checks, MPLS and LDP validation, inet.3 table inspection, and ping tests between edge routers (R1 and R6), confirmed that the network was correctly configured and fully operational. Overall, the project achieved its objective of

simplifying complex network deployment through automation.

10. REFERENCES

[1] Day one MPLS for Enterprise:

<https://www.juniper.net/documentation/en_US/day-one-books/MPLS4EEs_finalv2.pdf >

[2]MPLS Overview | Junos OS | Juniper Networks:

<https://www.juniper.net/documentation/us/en/software/junos/mpls/topics/topic-map/mpls-overview.html>

[3] Basic MPLS Configuration | Junos OS | Juniper Networks:

<https://www.juniper.net/documentation/us/en/software/junos/mpls/topics/topic-map/mpls-basic configuration.html>

[4] Ansible for Junos OS Developer Guide

<https://www.juniper.net/documentation/us/en/software/junos-ansible/ansible/ansible.pdf> 18